Optimizing DL Algorithms on Embedded Systems (Case Studies)

Sparsh Mittal

Acronyms

- DNN – Deep Neural Network
- CUDA – Compute unified device architecture
- OpenCL – Open computing language
- FPS – Frame per second
- CONV – Convolution
- OpenMP – Open multi-processing
- RCNN – Recurrent neural network

Low-Power Image Recognition Challenge

Low-Power Image Recognition

- **Challenge:** process 20,000 images in 10 min.
- If X% less images processed in 10 min, then mAP reduced by X%.
- If all images processed before 10min, the energy consumed till completion time is measured.
- FinalScore = mAP/EnergyConsumed.

- We will discuss Kang et al.'s entry which won 1st prize.

Kang et al., "Joint Optimization of Speed, Accuracy, and Energy for Embedded Image Recognition Systems", DATE 2018

Choice of Hardware Platform

GPU	Chip	Clock	GFlops
Mali-T628 MP6	Exynos 5422	533MHz	102.4
Mali-G71 MP18	Exynos 8895	546MHz	371.2
Adreno 540	Snapdragon 835	710MHz	567
Maxwell Cores x 256	Tegra X1	1000MHz	512
Pascal Cores x 256	Tegra Parker	1465MHz	750

- Jetson TX2 (Parker) chosen since
  - ➢It provides highest throughput.
  - ➢Many DNN kernels are already available thru cuDNN.
  - ➢For other devices, OpenCL kernels need to be developed.

Choice of Object Detection Algorithm

Model (Framework)	FPS	mAP
SSD (Caffe)	3.5	0.43
YOLOv2 (Darknet)	5.81	0.51
Tiny YOLO (Darknet)	17.4	0.32

- YOLO versions are better than SSD.
- Tiny YOLO better than YOLOv2 since its product of FPS and mAP is higher. So, it is chosen.
- Not all predictions of Tiny YOLO are accurate, so post-processing step (NMS) is run on CPU to select predictions with high threshold.

NMS – Non-maximum suppression

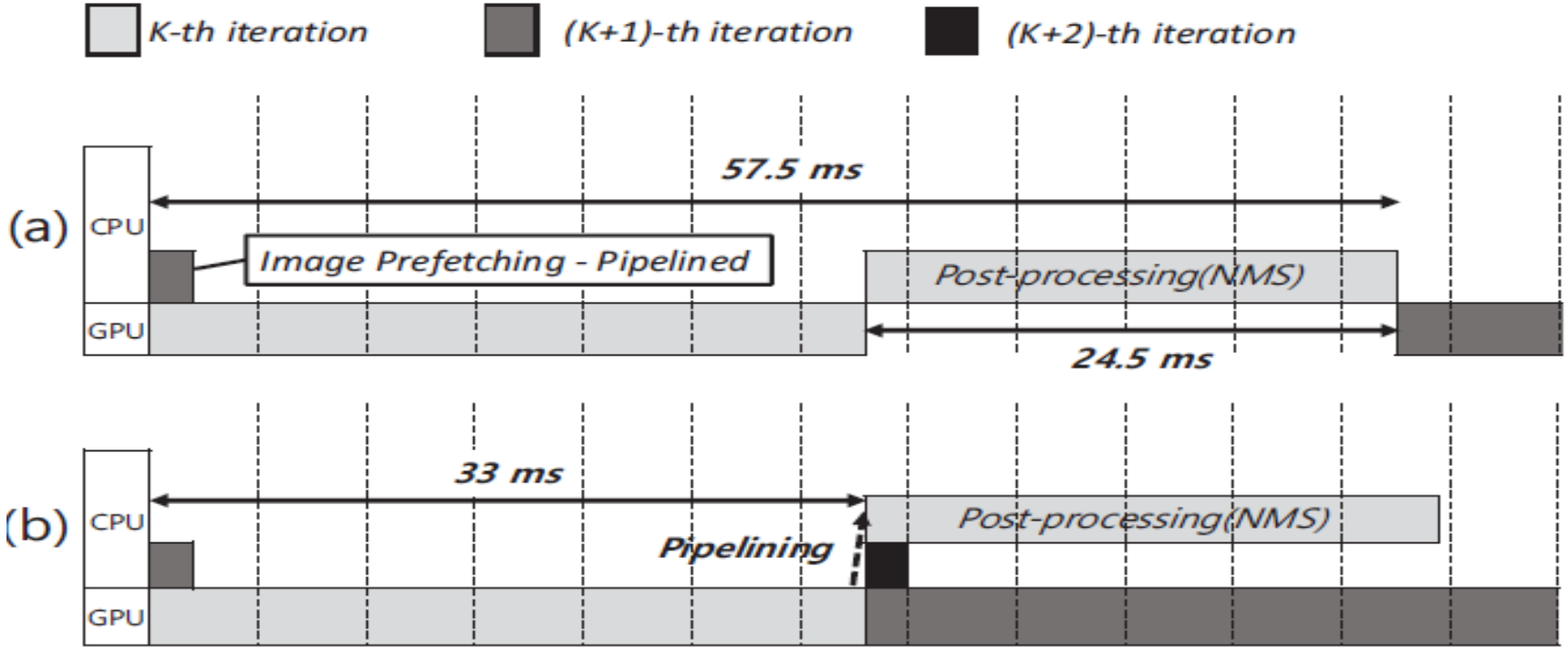
Optimizations Applied

- They apply a series of optimizations which are discussed next
- Their impact is summarized below

Step	Running Time	FPS	Improvement ratio
Baseline	1150s	17.4	1.0x
Pipelining	660s	30.3	1.74x
Tucker	540s	37.0	
Selective-Tucker	530s	37.7	1.25x
CPU Parallelization	502s	39.8	1.06x
16-bit Quantization	502s	39.8	
Selective-Quantization	460s	43.5	1.09x

Optimization 1: CPU-GPU Pipelining

- Use a buffer to store results of each iteration



	Inference Time	CPU Time	GPU Time
Baseline	1150 sec	488 sec	656 sec
Pipelined	660 sec	488 sec	656 sec

Kang et al., "Joint Optimization of Speed, Accuracy, and Energy for Embedded Image Recognition Systems", DATE 2018

Optimization 2: Tucker Decomposition

- Tucker is an approximate computing technique to accelerate CONVs. It reduces # of multiplications by using 1x1 convolution.
- Challenge: Tucker is not always useful.

Layer-wise profiling of execution time

	Baseline	Tucker
Conv1	120	120
Conv2	62	77
Conv3	34	44
Conv4	26	25
Conv5	23	21
Conv6	25	18
Conv7	90	46
Conv8	171	72
Conv9	27	26

Optimization 2: Tucker Decomposition

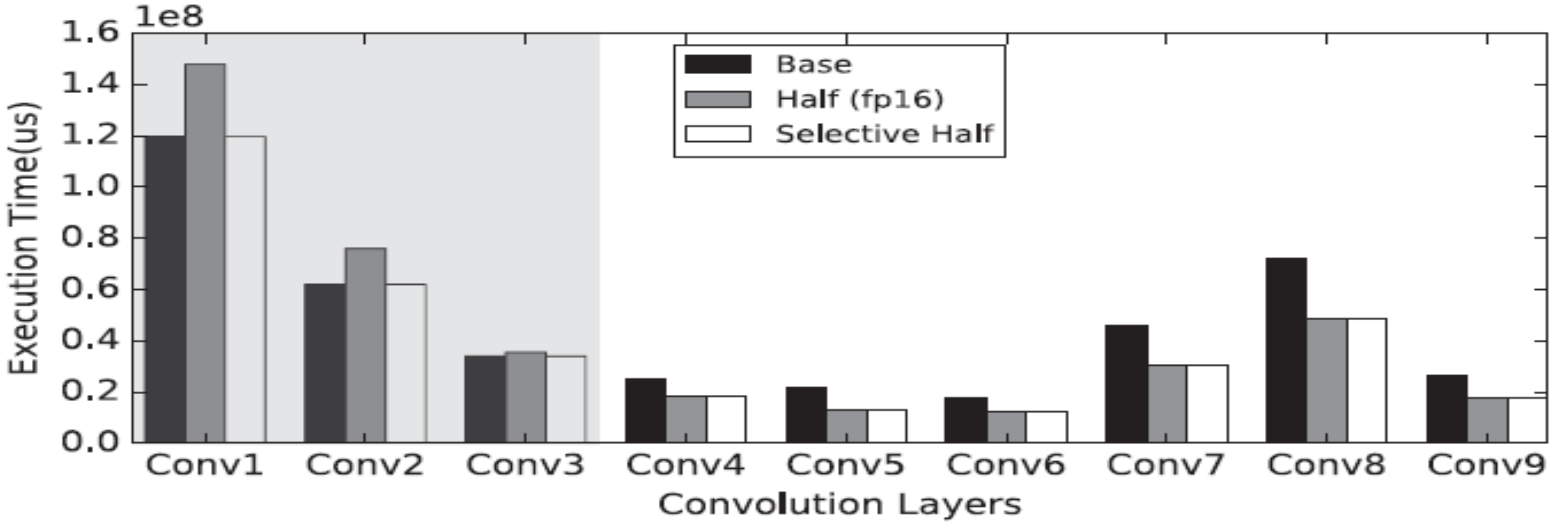
- Idea: use Tucker selectively in some layers

Layer-wise profiling of execution time

	Baseline	Tucker	Selective-Tucker
Conv1	120	120	120
Conv2	62	77	62
Conv3	34	44	34
Conv4	26	25	25
Conv5	23	21	21
Conv6	25	18	18
Conv7	90	46	46
Conv8	171	72	72
Conv9	27	26	26

Optimizations Three and Four: CPU Parallelization and Quantization

- To boost NMS, apply OpenMP parallelization on CPU.
- On GPU use 16b floating point instead of 32b floating.
- Challenge: some layers became slow after quantization due to increased number of instructions.



- Idea: apply quantization selectively in some layers

Kang et al., "Joint Optimization of Speed, Accuracy, and Energy for Embedded Image Recognition Systems", DATE 2018

Optimization 5: Frequency Scaling

- Previous optimizations reduce inference time to 460s.
- A slack of 140s is available.
- => Reduce CPU/GPU frequency to save energy.

This setting is chosen finally

CPU Freq.	GPU Freq.	Exec. Time(s)	Energy Consumption(Wh)
1,114MHz	1,122MHz	58	0.180682
1,728MHz	1,122MHz	52	0.184183
1,728MHz	944MHz	59	0.184728
1,114MHz	1,301MHz	55	0.192547
1,728MHz	1,301MHz	49	0.195594
2,035MHz	1,122MHz	52	0.196636
2,035MHz	944MHz	58	0.196706
2,035MHz	1,301MHz	48	0.205224

Driver Drowsiness Detection

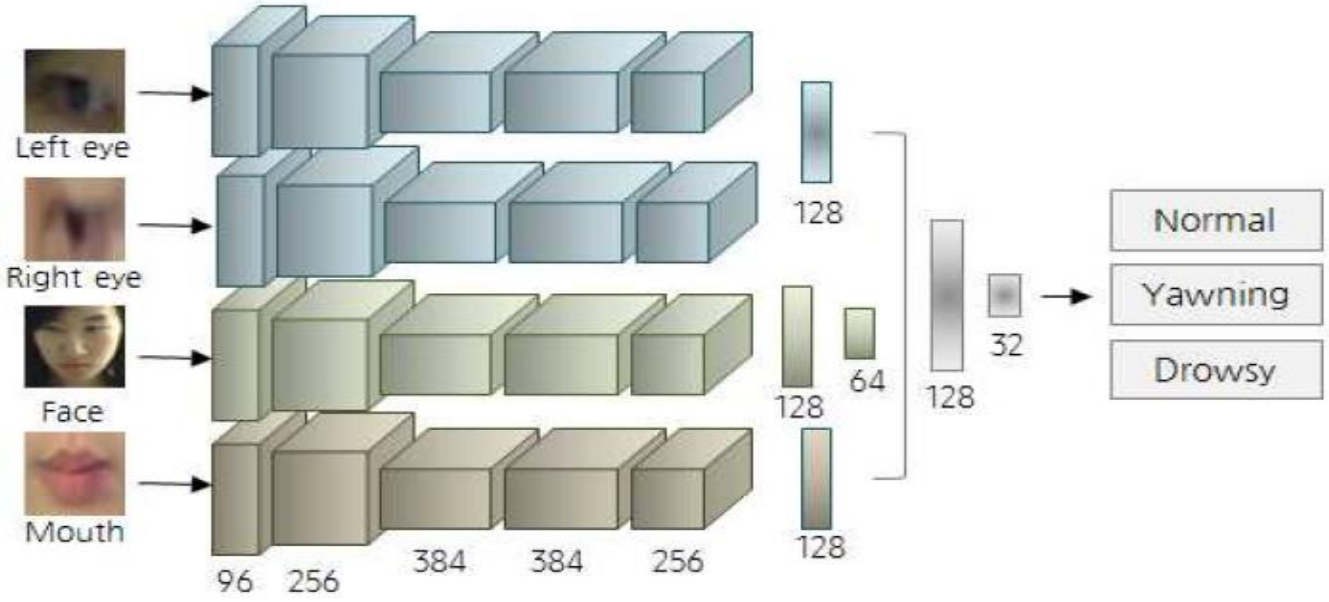
Images are courtesy of following paper: Reddy et al., “Real-time Driver Drowsiness Detection for Embedded System Using Model Compression of Deep Neural Networks, CVPRW, 2017

Goal and Summary of Design

- **Goal:** classify image of driver's face into one of 3 states: normal, yawning, drowsy.
- Step1: perform **face detection and alignment** using *multi-task cascaded CNN*.
- This provides coordinates of face boundary and locations of: left/right-eye, nose, left/right-lip-end.
- Step 2: detect driver drowsiness.
- Three models are proposed:
  - Baseline-4 model
  - Baseline-2 model
  - Compressed-2 model

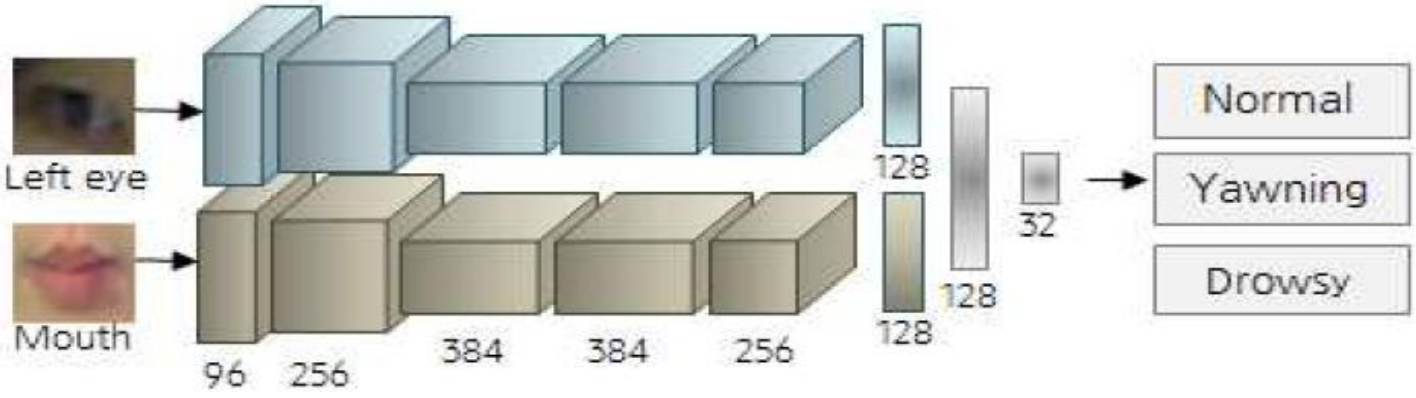
Baseline-4 Design

- Inputs to CNN: left-eye, right-eye, mouth and face.
- Each stream fed to a separate CNN, similar to AlexNet.
- CONV layers of eyes share same weight.



Baseline-2 Design

- Goal: reduce the number of streams to improve speed.
- Idea 1: for face recognition, features of partial face image are comparable to that of full-face image.
- Idea 2: movement of eyes is similar.
- Idea 3: mouth image: important to detect drowsiness.
- **Baseline-2 model** takes only left-eye and mouth.
- Accuracy degraded by 1 percent; speed doubled!



Compressed-2 model

- Use knowledge distillation.
- Teacher network: original large network, trained from the dataset itself. Baseline-2 used as teacher.
- Student network: smaller and faster network and it learns features from *soft targets* generated by the teacher network. Compressed-2 used as student.
- Both networks use the same filter sizes, but Compressed-2 network uses smaller number of kernels in CONV layers.

Hinton et al., "Distilling the knowledge in a neural network", NIPS, 2014

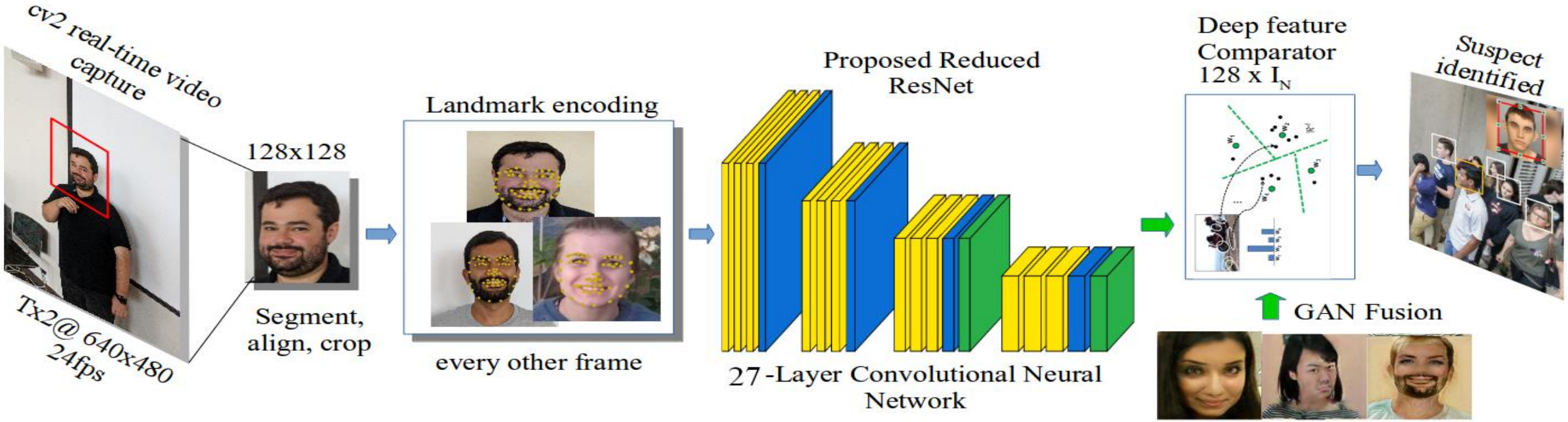
Results

	Model size (MB)		Accuracy	End-to-end speed (FPS)	
	On disk	During execution		GTX 1080	TK1
Baseline-4	56	600	91.30%	72	6.1
Baseline-2	28	443	93.80%	82	12.5
Compressed-2	10	353	89.50%	90	14.9
Faster RCNN	236	845	82.80%	22.7	1.1

Identifying faces of suspected people from CCTV footage

Images are courtesy of following paper: Sadique et al., “Reduced Residual Nets (Red-Nets): Low Powered Adversarial Outlier Detectors”, IRI, 2018

Block Diagram

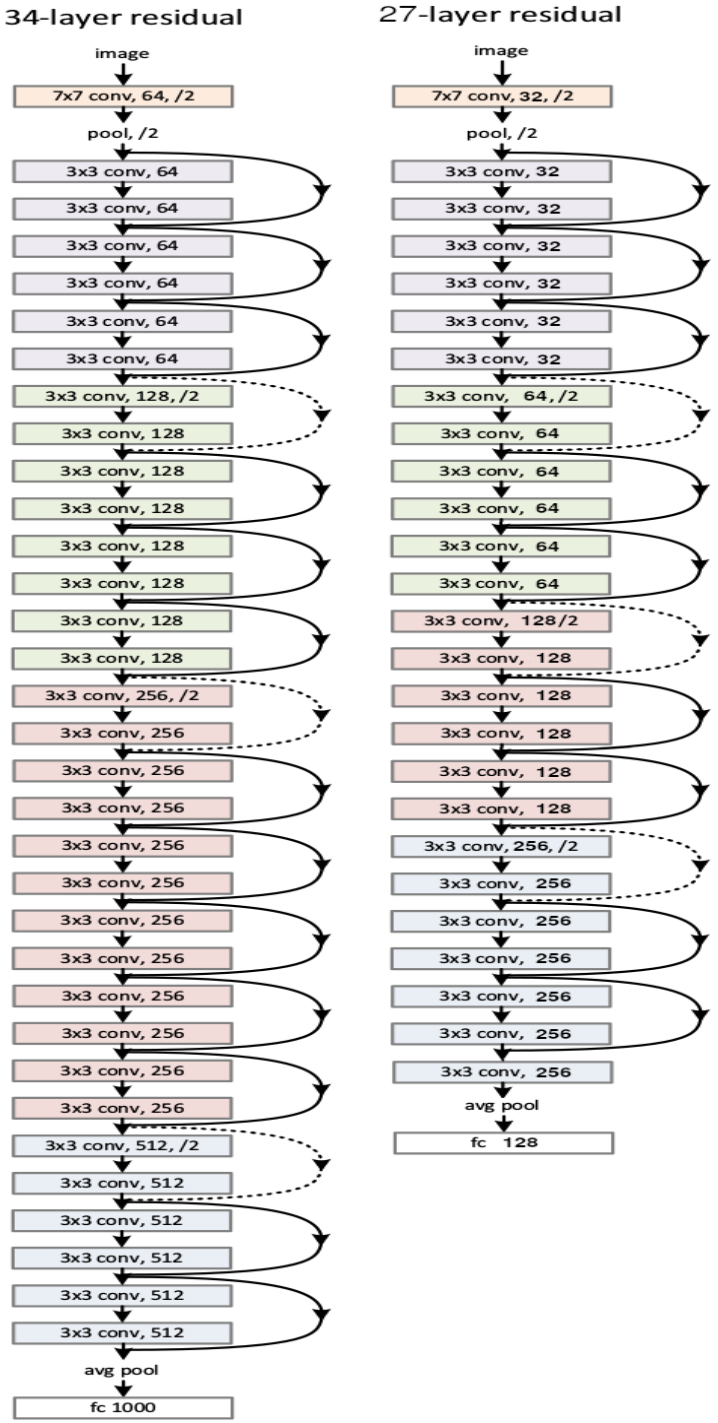


- Scale 640x480 images to 128x128 faces
- Encode each face as a 128-bit vector

CNN Simplification

- Instead of 34 or 50-layer ResNet, use 27-layer ResNet => less compute

layer name	output size	27-layer	34-layer	50-layer
conv1	112x112	7x7,32, stride 2	7x7,64,stride 2	
conv2x	56x56	3x3 max pool, stride 2		
		[3x3,32] x6	[3x3,64] x6	[1x1,64] [3x3,64] [1x1x256] x3
conv3x	28x28	[3x3,64] x6	[3x3,128] x8	[1x1,128] [3x3,128] [1x1x512] x4
conv4x	14x14	[3x3,128] x6	[3x3,256] x12	[1x1,256] [3x3,256] [1x1x1024] x6
conv5x	7x7	[3x3,256] x7	[3x3,512] x6	[1x1,512] [3x3,512] [1x1x2048] x3
	1x1	acg pool, 128-d fc, softmax	acg pool, 1000-d fc, softmax	
FLOPS		2.8×10^9	3.6×10^9	3.8×10^9



Optimizations

- Compared to 34-layer, 27-layer ResNet reduces
  - Number of filters per layer by 1/2.
  - Dimension of pooling layer from 1000 to 128.

- Challenge: Minor perturbations to even few features of face can confuse traditional image classifiers.
- This weakness can be exploited by changing the facial appearance.

Using Generative Adversarial Networks (GAN)

- GANs generate realistic adversarial images that can be used for robust training of facial recognition system.

Adversarially generated images to capture key markers of target faces

- In GAN, generator produces data and discriminator utilizes this data for ascertaining authenticity of input.
- Use *adversarial loss* for making the generated images identical to the real images.
- Encode adversarial images in same manner and fuse with the deep feature comparator to detect suspects.

Results

- On TX2, 99.4 % accuracy in finding whether two facial images are from the same person.
- Only 17.6 Watt-hour power.